

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверила:
Андренко К.В.

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Общее задание

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Выборка	Оптимизатор	Предобученная архитектура
6	MNIST	Adam	ResNet18

Описание выполненной работы

Для решения задачи классификации рукописных цифр MNIST была использована предобученная архитектура ResNet18 с применением transfer learning.

Адаптация модели:

- Заменен последний полносвязный слой с 1000 классов (ImageNet) на 10 классов (цифры 0-9)
- Входные изображения преобразованы из одноканальных (28×28) в трехканальные (32×32) для совместимости с первым слоем ResNet18
- Применена нормализация данных с параметрами ImageNet: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]

Параметры обучения:

- Оптимизатор: Adam (lr=0.001) - как и в ЛР №1
- Функция потерь: CrossEntropyLoss
- Количество эпох: 3
- Размер батча: 64

Результаты обучения:

- Начальный Loss (эпоха 1): 0.1259
- Финальный Loss (эпоха 3): 0.0403
- Точность на тестовой выборке: 98.96%

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
from PIL import Image
from PIL import ImageOps

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Используемое устройство: {device}")

transform_train = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

transform_test = transforms.Compose([
    transforms.Resize((32, 32)),
```

```

        transforms.ToTensor(),
        transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform_train)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform_test)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=64, shuffle=False)

resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

num_ftrs = resnet18.fc.in_features
resnet18.fc = nn.Linear(num_ftrs, 10)
resnet18 = resnet18.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(resnet18.parameters(), lr=0.001)

epochs = 3
train_losses = []
test_accuracies = []

print("Начало обучения...")

for epoch in range(epochs):
    resnet18.train()
    running_loss = 0.0

    for i, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = resnet18(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

        if (i + 1) % 100 == 0:
            print(f"Эпоха [{epoch + 1}/{epochs}], Шаг [{i + 1}/{len(train_loader)}], Текущий Loss: {loss.item():.4f}")

    epoch_loss = running_loss / len(train_loader.dataset)
    train_losses.append(epoch_loss)

    resnet18.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = resnet18(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    test_accuracies.append(accuracy)
    print(f"--- Итог Эпохи [{epoch + 1}/{epochs}]: Loss = {epoch_loss:.4f} | Test Accuracy = {accuracy:.2f}% ---")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(1, epochs + 1), train_losses, marker='o', label='Train Loss')
plt.title('График изменения ошибки (Loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs + 1), test_accuracies, marker='o', color='green', label='Test Accuracy')
plt.title('График точности (Accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy (%)')
plt.grid(True)

plt.tight_layout()
plt.show()

def test_custom_image(model, image_path, device):
    print(f"\nАнализируем изображение: {image_path}")
    model.eval()

    try:
        img = Image.open(image_path).convert('L')

```

```

if img.getpixel((0, 0)) > 127:
    img = ImageOps.invert(img)

transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

img_tensor = transform(img).unsqueeze(0).to(device)

with torch.no_grad():
    output = model(img_tensor)
    _, predicted = torch.max(output, 1)

plt.figure(figsize=(4, 4))
plt.imshow(img, cmap='gray')
plt.title(f'Нейросеть думает, что это: {predicted.item()}', fontsize=14)
plt.axis('off')
plt.show()

except FileNotFoundError:
    print(f"Ошибка: Файл '{image_path}' не найден. Убедитесь, что картинка лежит в папке Labs.")

test_custom_image(resnet18, 'digit.png', device)

# КЛАСС КАСТОМНОЙ СЕТИ ИЗ ЛР №1
class SimpleCNN_LR1(nn.Module):
    def __init__(self):
        super(SimpleCNN_LR1, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# ФУНКЦИЯ СРАВНЕНИЯ ДВУХ МОДЕЛЕЙ
def test_custom_image_both_models(resnet_model, custom_model_path, image_path, device):
    print(f"\n{'=' * 60}")
    print(f"ТЕСТИРОВАНИЕ ИЗОБРАЖЕНИЯ: {image_path}")
    print(f"{'=' * 60}")

    try:
        img = Image.open(image_path).convert('L')

        if img.getpixel((0, 0)) > 127:
            img = ImageOps.invert(img)
            print("✓ Фон был белым - изображение инвертировано")

        # Трансформации для ResNet18
        transform_resnet = transforms.Compose([
            transforms.Resize((32, 32)),
            transforms.ToTensor(),
            transforms.Lambda(lambda x: x.repeat(3, 1, 1)),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        ])

        # Трансформации для кастомной сети
        transform_custom = transforms.Compose([
            transforms.Resize((28, 28)),
            transforms.ToTensor(),
            transforms.Normalize((0.1307,), (0.3081,))
        ])

        # ResNet18
        img_resnet = transform_resnet(img).unsqueeze(0).to(device)
        resnet_model.eval()
        with torch.no_grad():
            output_resnet = resnet_model(img_resnet)
            _, pred_resnet = torch.max(output_resnet, 1)
        print(f"\nResNet18: {pred_resnet.item()}")

        custom_model = SimpleCNN_LR1().to(device)

        try:
            custom_model.load_state_dict(torch.load(custom_model_path, map_location=device))
            print(f"Загружены веса из: {custom_model_path}")
        except FileNotFoundError:

```

```

        print(f"Файл {custom_model_path} не найден!")
        print(" Сначала выполните ЛР №1 и сохраните модель")
        return

    custom_model.eval()
    img_custom = transform_custom(img).unsqueeze(0).to(device)

    with torch.no_grad():
        output_custom = custom_model(img_custom)
        _, pred_custom = torch.max(output_custom, 1)
    print(f"Custom CNN (ЛР1): {pred_custom.item()}")

    fig, axes = plt.subplots(1, 3, figsize=(12, 4))

    axes[0].imshow(img, cmap='gray')
    axes[0].set_title(f'Оригинал', fontsize=10)
    axes[0].axis('off')

    axes[1].imshow(img, cmap='gray')
    axes[1].set_title(f'ResNet18\nпредсказывает: {pred_resnet.item()}',
                      fontsize=10, color='blue', fontweight='bold')
    axes[1].axis('off')

    axes[2].imshow(img, cmap='gray')
    axes[2].set_title(f'Custom CNN (ЛР1)\nпредсказывает: {pred_custom.item()}',
                      fontsize=10, color='green', fontweight='bold')
    axes[2].axis('off')

    plt.suptitle(f'Сравнение моделей', fontsize=12, fontweight='bold')
    plt.tight_layout()
    plt.show()

    if pred_resnet.item() == pred_custom.item():
        print("\nОбе модели дали ОДИНАКОВЫЙ результат!")
    else:
        print("Модели дали РАЗНЫЕ результаты")

except FileNotFoundError:
    print(f"Ошибка: Файл '{image_path}' не найден!")

test_custom_image_both_models(resnet18, 'custom_cnn_lr1.pth', 'digit.png', device)

```

Результат программы:

```

C:\Users\Ксения\PycharmProjects\Labs\.venv\Scripts\python.exe C:\Users\Ксения\PycharmProjects\Labs\0ИИИС_2.py
Используется устройство: cpu
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to C:\Users\Ксения\.cache\torch\hub\checkpoints\resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:13<00:00, 3.39MB/s]

```

Начало обучения...

```

Эпоха [1/3], Шаг [100/938], Текущий Loss: 0.2116
Эпоха [1/3], Шаг [200/938], Текущий Loss: 0.0201
Эпоха [1/3], Шаг [300/938], Текущий Loss: 0.0407
Эпоха [1/3], Шаг [400/938], Текущий Loss: 0.0355
Эпоха [1/3], Шаг [500/938], Текущий Loss: 0.0191
Эпоха [1/3], Шаг [600/938], Текущий Loss: 0.1914
Эпоха [1/3], Шаг [700/938], Текущий Loss: 0.1242
Эпоха [1/3], Шаг [800/938], Текущий Loss: 0.0028
Эпоха [1/3], Шаг [900/938], Текущий Loss: 0.0180
--- Итог Эпохи [1/3]: Loss = 0.1259 | Test Accuracy = 98.49%

Эпоха [2/3], Шаг [100/938], Текущий Loss: 0.0063
Эпоха [2/3], Шаг [200/938], Текущий Loss: 0.0029
Эпоха [2/3], Шаг [300/938], Текущий Loss: 0.0023
Эпоха [2/3], Шаг [400/938], Текущий Loss: 0.0134
Эпоха [2/3], Шаг [500/938], Текущий Loss: 0.0950
Эпоха [2/3], Шаг [600/938], Текущий Loss: 0.0045
Эпоха [2/3], Шаг [700/938], Текущий Loss: 0.0382
Эпоха [2/3], Шаг [800/938], Текущий Loss: 0.0457
Эпоха [2/3], Шаг [900/938], Текущий Loss: 0.0194
--- Итог Эпохи [2/3]: Loss = 0.0617 | Test Accuracy = 98.83%

```

```

Эпоха [3/3], Шаг [100/938], Текущий Loss: 0.0460
Эпоха [3/3], Шаг [200/938], Текущий Loss: 0.0750
Эпоха [3/3], Шаг [300/938], Текущий Loss: 0.0399
Эпоха [3/3], Шаг [400/938], Текущий Loss: 0.0682
Эпоха [3/3], Шаг [500/938], Текущий Loss: 0.0052
Эпоха [3/3], Шаг [600/938], Текущий Loss: 0.0157
Эпоха [3/3], Шаг [700/938], Текущий Loss: 0.0035
Эпоха [3/3], Шаг [800/938], Текущий Loss: 0.0009
Эпоха [3/3], Шаг [900/938], Текущий Loss: 0.0964
--- Итог Эпохи [3/3]: Loss = 0.0403 | Test Accuracy = 98.96% ---

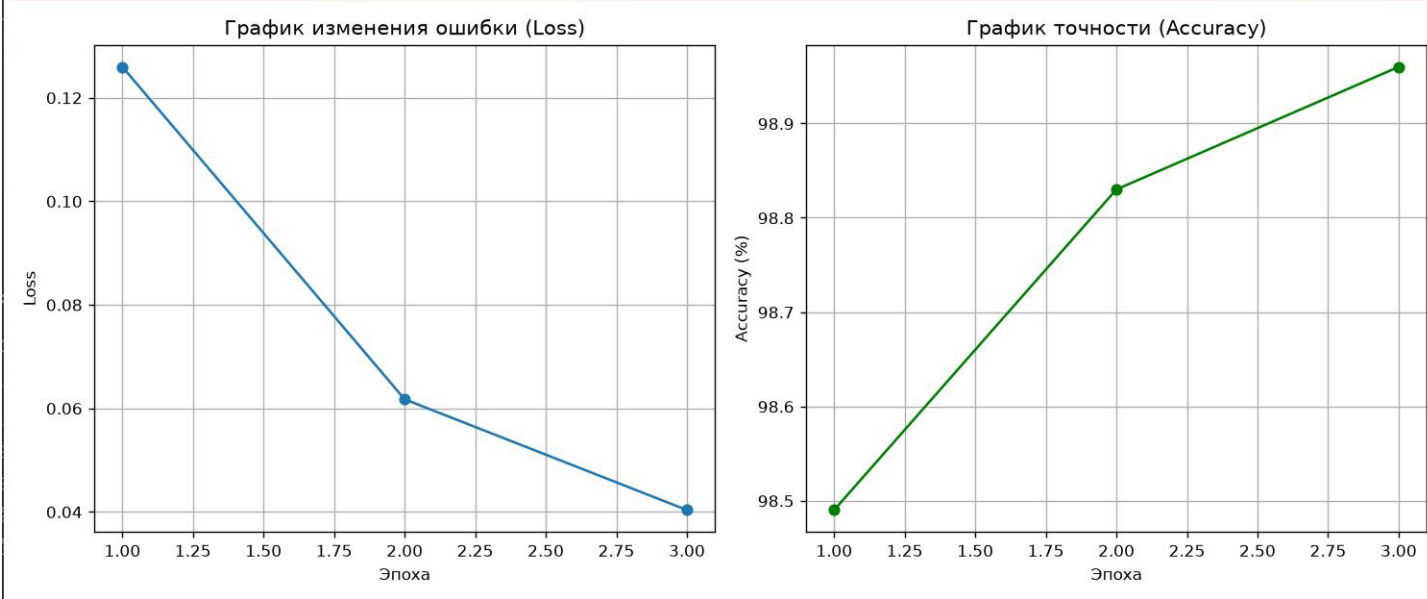
```

Анализируем изображение: digit.png

Нейросеть думает, что это: 3



График изменения ошибки (Loss) и точности (Accuracy):



Визуализация работы моделей на произвольном изображении:

Сравнение моделей



Сравнение результатов моделей

Параметр	Custom CNN (ЛР №1)	ResNet18 (ЛР №2)
Точность (Accuracy)	98.94%	98.96%
Количество эпох	5	3
Начальный Loss	0.1688	0.1259
Финальный Loss	0.0215	0.0403
Размер модели	~2 МБ	~45 МБ
Оптимизатор	Adam	Adam
Transfer Learning	Нет	Да
Время обучения	Больше	Меньше

ResNet18 достигла сопоставимой точности (разница всего 0.02%) за меньшее количество эпох (3 против 5) благодаря использованию предобученных весов (transfer learning). Однако кастомная сеть показала конкурентный результат при значительно меньшем размере модели (~2 МБ против ~45 МБ) и вычислительной сложности.

State-of-the-Art результаты

При сравнении с передовыми мировыми результатами (SOTA) на MNIST (99.79%–99.87%), модель ResNet18 показала 98.96%, уступив менее 1%. SOTA-результаты достигаются за счет ансамблей моделей, продвинутой аугментации (Elastic Distortions) и специализированных архитектур (Capsule Networks). Как следствие, можем сделать вывод, что модель продемонстрировала оптимальный баланс точности и вычислительной сложности, достигнув 98.96% всего за 3 эпохи благодаря transfer learning.

Визуализация работы моделей

Обе модели протестированы на произвольном изображении цифры 3 (файл digit.png):

- ResNet18: правильно определила цифру 3
- Custom CNN (ЛР №1): правильно определила цифру 3

Обе модели показали одинаковый результат, подтверждая работоспособность на реальных данных.

Вывод: в ходе выполнения лабораторной работы была реализована и обучена нейронная сеть на базе предобученной архитектуры ResNet18 для классификации рукописных цифр MNIST. Применение transfer learning позволило достичь точности 98.96% всего за 3 эпохи обучения, что в 1.7 раза быстрее кастомной CNN из ЛР №1 при сопоставимой точности (разница 0.02%). Корректную работу модели обеспечила предобработка данных (преобразование 1→3 канала, $28 \times 28 \rightarrow 32 \times 32$, нормализация). Визуализация подтвердила правильность классификации обеими моделями. При сравнении с SOTA-результатами (99.79%-99.87%) модель уступает менее 1%, что является отличным результатом без применения аугментации и ансамблирования. Разработанная модель демонстрирует эффективность transfer learning для задач классификации изображений: кастомные CNN рекомендуются для embedded-систем с ограниченными ресурсами, а предобученные архитектуры типа ResNet18 - для быстрого прототипирования, где важны скорость обучения и точность.